

Distributed Systems - 8_10

Today we'll see the failure detectors. Once we have a failure detector, we don't care about the problem of synchrony on a system, we can solve problems even if channels are asynchronous.

Let's recap on the timing assumptions.

In synchronous systems, we can build a global common clock (we have time assumptions), and there is a bound.

In asynchronous we use the ordering with logical/vector clock.

In the eventually synchronous systems we have a time T , unknown to us, after it the system becomes synchronous.

Proving something in a system like this could not be easy.

We have two choices:

1. Put assumptions on the system model
2. Create separate abstractions (in the failure detection family)

We'll have that the entire time is encapsulated inside this assumption.

The failure detector has to detect a failure, and only works for crash failure.

They are described by two properties:

- Accuracy: accuracy in detecting a failure, we're accurate if, when we say that p failed, p actually failed.
- Completeness, force the failure detector to detect the failure sometimes.

The stronger are the timing assumptions, the more accurate the properties are.

In a synchronous system, we can always build a failure detector that is always accurate and concrete. In an async system, it's a bit different, some properties will not be as strong.

It's based on the idea of pinging processes. By replies we collect data, and he doesn't reply after a certain time we say that the process is dead.

When a process crashes, it doesn't recover.

We start by taking a look at P , **perfect failure detectors**.

We'll build P in synchronous systems, with crash failures, and perfect synchronous point-to-point. We also have another property that, if a message is sent at time t , it's received at max at $t + \Delta$.

Using the clock and the bound, we can detect if someone has crashed.

Obviously this strategy cannot work in asynchronous systems, in fact there is no way to reliable way to detect failure in async systems.

Let's see the abstraction.

The P is a box, with a single interface with the upper layer, given by $Crash(p_x)$. It will run on our systems, and then at a certain time it will create an event on you saying that a process p_x has crashed.

What are the properties?

- Strong completeness property: eventually every process that crashes is permanently detected by every correct process. *If a process crashes, after some time, all the processes that are correct, will signal the crash. Thanks to this, we can't build a failure detector that is silent, so it's complete.*
- Strong accuracy: if a process p is detected by any process, then p has crashed.

Algorithm 2.5: Exclude on Timeout

Implements:

PerfectFailureDetector, **instance** $\mathcal{P}$.

Uses:

PerfectPointToPointLinks, **instance** pl .

upon event $\langle \mathcal{P}, Init \rangle$ **do**

$alive := \Pi$;
 $detected := \emptyset$;
 $starttimer(\Delta)$;

upon event $\langle Timeout \rangle$ **do**

forall $p \in \Pi$ **do**
 if $(p \notin alive) \wedge (p \notin detected)$ **then**
 $detected := detected \cup \{p\}$;
 trigger $\langle \mathcal{P}, Crash \mid p \rangle$;
 trigger $\langle pl, Send \mid p, [HEARTBEATREQUEST] \rangle$;
 $alive := \emptyset$;
 $starttimer(\Delta)$;

upon event $\langle pl, Deliver \mid q, [HEARTBEATREQUEST] \rangle$ **do**

trigger $\langle pl, Send \mid q, [HEARTBEATREPLY] \rangle$;

upon event $\langle pl, Deliver \mid p, [HEARTBEATREPLY] \rangle$ **do**

$alive := alive \cup \{p\}$;

The perfect p2p is built with the synchronous property.

We have to ping a process infinitely often, periodically with Δ .

In the init we have a set of alive processes, with everyone alive at first. In the detected set, we put processes that we detected that are crash. To conclude, we start a timer.

The third handler is just the ping/pong handler.

The fourth receives a message and see that someone has sent the pong, and updates the alive set.

The timeout handler works at the end of every Δ . For each process in Π , we send a message of ping to everyone. We also check if the process is not in alive or detected set, because in that case the process is dead. After each loop, we set alive to $\emptyset$, because we need to check again if the processes are alive, and thus we need to update the set.

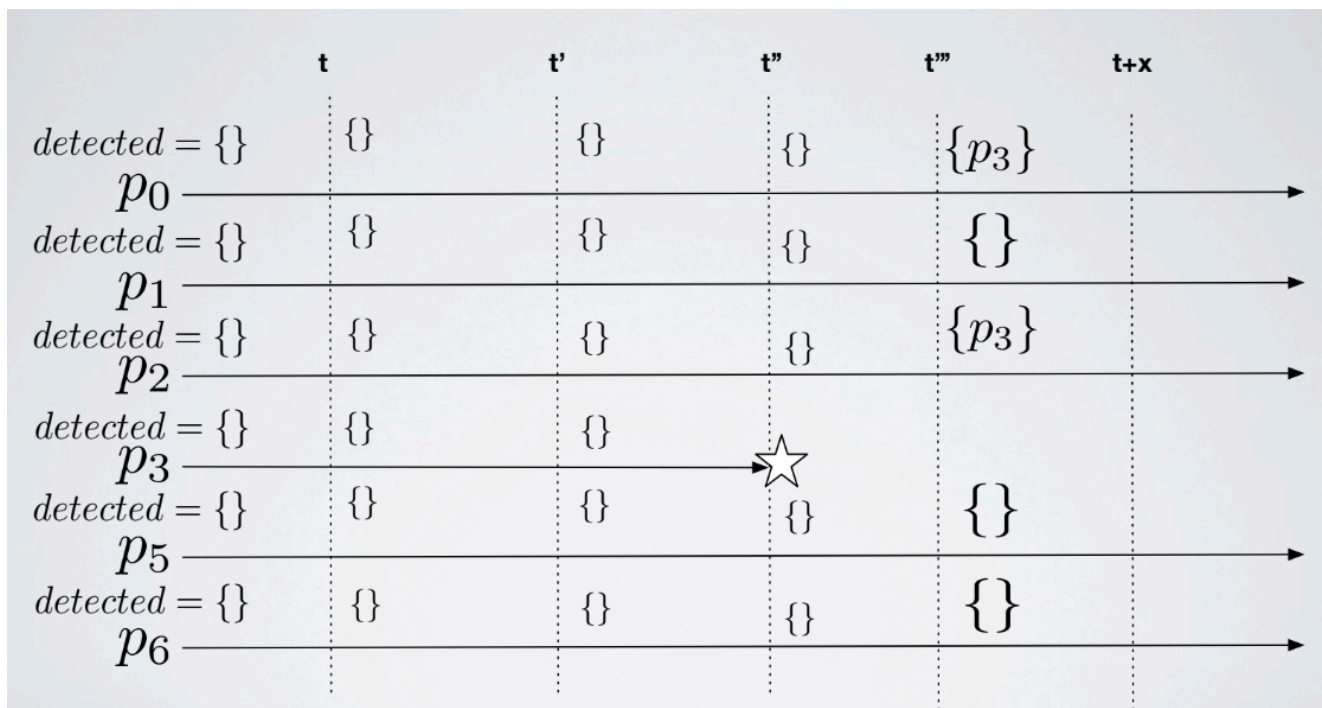
Now let's show if this works.

Strong completeness: a process p_1 crashes at time t , then p_0 detects the crash at time $t + 2\Delta$.

...

For Strong accuracy. By contradiction. We have to do an assumption, we have to assume that $\Delta \geq RTT$. Let's suppose that p_1 raise a flag saying that p_2 is dead, but p_2 is alive. The only reason this can happen is that we send a ping, but the pong arrives after Δ . This is not possible because the ping will arrive in $\frac{RTT}{2}$, but also the pong will arrive in $\frac{RTT}{2}$ (it has to arrive in time), and so the assumption is not possible.

Let's now take a look at an execution of a P .



Is it possible? We don't have to reason with the implementation, but with the properties. At the moment yes, because in order to violate it we would have that the processes will never detect that p_3 has failed. It's not telling us anything about the latency.

Is it possible that P is built with a modification of strong completeness:

- Strong completeness: Eventually, every process that crashes is permanently detected by every ~~correct~~ process.

No, because if we as a process have crashed, how can we detect that in the future a property will crash in the future?

The algorithm (named "Exclude on Timeout"), if a certain time t i dont' have a process p in my detected set, can I state that p is alive at time t ? No, we are not sure about that. We only know the world at time $t - \Delta$, so no failure detector can tell if someone is alive at time t , there's a delay.

What is the complexity of algorithm P . In a Δ time, how many messages are generated? $2n^2$, because each process send n messages to check and receives n messages, so $2n$, and then we have n processes.

Can we cut the message by half? Yes, we could check the Ping coming from other processes, which means that the process is still alive.

We divide the time in rounds, and at the beginning of the round everyone will send a Heartbeat saying that they are alive. If a process doesn't send a ping to start with, then it's silent and thus the process is dead.

The idea will be a bit more complicated. We know that in real life, even if clocks are synchronized, there is a skew between them.

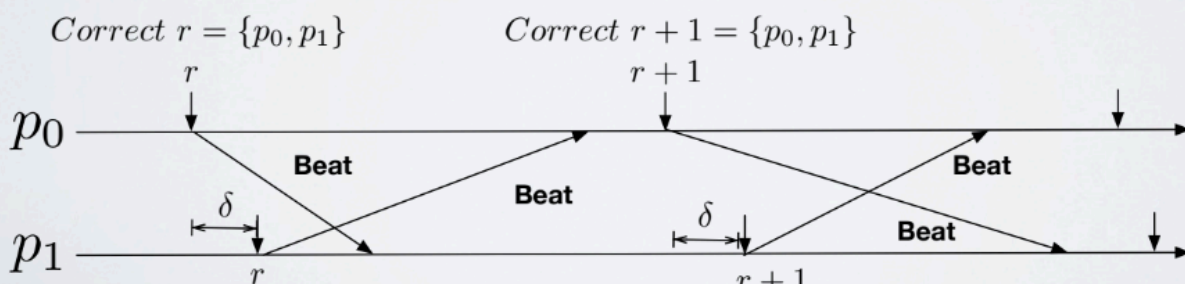
Algorithm 3 Round-Based, Failure Detector $\mathcal{P}$ on process p_i

```

1: upon event INIT
2:    $C_v$ 
3:    $T$ 
4:    $Corrects = \{p_0, p_1, p_2, \dots, p_n\}$ 
5:    $RoundAlive = Corrects$ 
6:    $r = -1$ 
7:   ▷ Actions to execute at each new round
8:   upon event  $C_v = kT$  FOR SOME  $k \in \mathbb{N}^+$ 
9:      $r = r + 1$ 
10:    for all  $p_j \in Corrects \setminus RoundAlive$  do
11:       $Corrects = Corrects \setminus \{p_j\}$ 
12:      TRIGGER  $\langle Crash | p_j \rangle$ 
13:       $RoundAlive = \{p_i\}$ 
14:      for all  $p_j \in Corrects$  do
15:        SEND( $\langle p_i, BEAT \rangle$ )
16: upon event DELIVERY( $\langle p_j, BEAT \rangle$ )
17:    $RoundAlive = RoundAlive \cup \{p_j\}$ 

```

▷ Synchronizer's clock
▷ Round length: $T > 2 \cdot (max_delay + \delta)$
▷ Myself



In the init we have a variable clock C_v , a number that is always increasing with the hardware clock. We also have a parameter T , the same for all processes, it's the length of the round. We have the set of correct processes (at the beginning it's everyone), and a set RoundAlive, that we update to remember who is alive and who's not. There is also a counter, initially set to -1, and we use it to take track of the round number.

How does the algorithm work? When the clock is equal to k multiplied by T , for $k \in \mathbb{N}$.

Let's say that T is 10 seconds, this handler will trigger at time 0, then with $k = 1$ at 10 second, with $k = 2$ at 20 seconds, and so on, it's a multiple of a interval. I increase the round number (starting with round 0, $-1 + 1 = 0$), and for all processes that are correct difference roundalive, we update their status by removing them from correct, and trigger a crash. At round 0 the trigger will not be "triggered" because Corrects and RoundAlive are the same. I then update roundalive equal to myself (i'm alive), and for all processes that are correct we send them a BEAT.

In the figure, p_1 starts later, after δ . When a process receives a beat, he updates the set of roundalive, and we continue.

If p_1 fails what happens? At the round $r + 1$ we send a beat to p_1 , but he is dead. When we trigger the loop, we'll have that corrects is given by p_0, p_1 , while roundalive is only given by p_0 . Thus I will remove p_1 from the correct set.

We have to know, in the proof, that the message is quick enough to reach us, even if there is a delay of δ .

We use two facts:

- If I send a beat i'm alive
- If I don't send it I'm dead

We have to show that if i send a beat in ...

We have two processes p_1 and p_2 . We have to show that the beat of p_2 when it reaches p_1 is also in round 2 (p_2 starts in round 2). Let's suppose that the beat reaches p_1 in another round, which could be with $r = 1$ or $r = 3$.

Let's say that $r < 2$. The worst scenario that p_1 starts round 2 after us (at the latest possible time), so at $t + \delta$ (with p_2 starting round two at t). What is the only way to reach us before this time? The message has to be quick, and the delay $< \delta$.

Remember that we are assuming that the delay is $> \delta$, then this thing cannot happen.

If the delay is $< \delta$, we can artificially delay messages (even if we receive it before δ , we wait to deliver it).

We now have to show that $r \not\geq 2$ (cannot be). Let's say that r at p_2 starts with 2 at time t , and at time t'' p_1 starts with $r = 3$, with $t'' > t$.

The worst possible case is that p_1 starts the round before p_2 , at time $t' = t - \delta$.

Then $t'' = t - \delta + T$. Then the interval between t'' and t is given by $T - \delta$. For the algorithm to be correct, we have to ensure that $T - \delta > \text{maximum delay}$. We can decide the value of T , with $T = 2 \cdot (MAX + \delta)$, so we have that

$2MAX + \delta > MAX$, which is obvious.

This ensures that we receive the message in the interval of $T - \delta$.

Now we have to prove the strong completeness. It's trivial, if a process dies he will not send a BEAT, and we can't use a BEAT coming from a previous round.

For strong accuracy, let's say by contradiction that in a round the process is dead, the only way for not receiving the beat is the process not sending it, and this happens only if the process is dead.

🔗 Exercise

Find the minimum time for the algorithm to work.

Why is this algorithm important? It shows us the equivalence between a sync system and a round based system.

Let's say now we have our round based P . Is it possible to have an execution like this?

	r=0	r=1	r=2	r=3	r=4
$detected = \{\}$ p_0	$\{\}$	$\{\}$	$\{\}$	$\{p_3\}$	$\{p_3\}$
$detected = \{\}$ p_1	$\{\}$	$\{\}$	$\{\}$	$\{\}$	$\{p_3\}$
$detected = \{\}$ p_2	$\{\}$	$\{\}$	$\{\}$	$\{p_3\}$	$\{p_3\}$
$detected = \{\}$ p_3	$\{\}$	$\{\}$			
$detected = \{\}$ p_5	$\{\}$	$\{\}$	$\{\}$	$\{\}$	$\{p_3\}$
$detected = \{\}$ p_6	$\{\}$	$\{\}$	$\{\}$	$\{\}$	$\{p_3\}$

Yes it is, p_3 might die in the middle of the for of sending beats, and only someone receives that the beat, while p_0 and p_2 not (for example). Even if processes are ordered in Π , the set of processes that receive the message is arbitrary (they might get stuck in the kernel, or something like that).

Let us go back at the Exclude on Timeout algorithm. What happen if we do an implementation that δ is the minimum? If δ is smaller than RTT, then we might not receive the beat from other processes. We'll build something that revise the set of "dead" processes, building an eventually perfect failure detector $\diamond P$.

It was built to work in an eventually synchronous system. It's an algorithm that will make mistakes at first, and then it will stabilize to the correct behavior forever when the system becomes synchronous.

The interface of the $\diamond P$ is different from P . ($\diamond$ is used, usually, for stabilizing algorithms)

It has two events:

- Suspect: "I believe that p_1 is dead, but I could be wrong"
- Restore: "Actually I made a mistake, p_1 is probably alive"

The properties:

- (Strong completeness): eventually every process that crashes is permanently suspected by every correct process.
- (Eventual strong accuracy) eventually no correct process is suspected by any correct process. *when the system stabilize, a process cannot be suspected*

anymore, we can't make anymore mistakes

These two properties ensure that the set of suspected stabilizes on a correct processes.

How could we implement this algorithm? We could use a ping and pong. When we make a mistake we double δ .

I didn't get the last part that much.